

LoRA & QLoRA

Low-Rank Adaptation (LoRA) :

is a parameter-efficient fine-tuning method for large pretrained models that keeps the original model weights frozen and learns only small, low-rank matrices (adapters) inserted into the network. By training these lightweight adapters instead of the full model, LoRA dramatically reduces the number of trainable parameters, speeds up fine-tuning, and makes it easier to store and deploy multiple task-specific versions of the same base model.

Why LoRA Exists:

Large models have billions of parameters. **Fully fine-tuning them is:**

- Expensive in GPU memory & compute

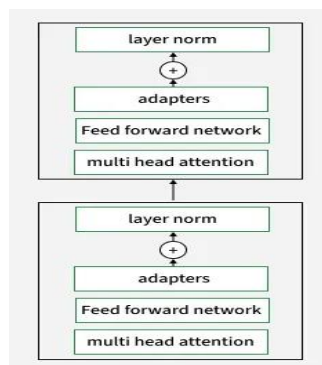
- Slow

- Hard to store multiple task-specific versions

LoRA solves this by only training a tiny set of additional parameters while keeping the original model frozen.

How LoRA Works :

This image shows a transformer block architecture where adapters are inserted after the feed forward network and before layer normalization. In LoRA, these adapters are implemented as low-rank matrices. During fine-tuning, only these adapter parameters are updated, while the core model weights (multi-head attention, feed-forward network, etc) stay fixed.



Adapters in the Stack: Each transformer block contains a small adapter module.

Parameter Update: Only the adapters (the low-rank matrices) are updated during training, greatly reducing the number of trainable parameters.

let's use some notation to understand the idea :

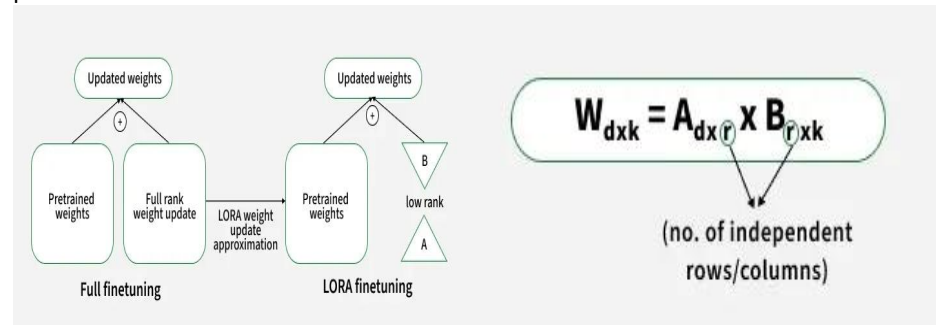
W_0 : all parameters of a pre-trained model in a matrix.

ΔW : additional weight changes.

The final weights are obtained by adding these two matrices: $W_0 + \Delta W$.

The key innovation of LoRA lies in decomposing the weight change matrix ΔW into two low-rank matrices, A and B. Instead of directly training the parameters in ΔW , LoRA focuses on training the parameters in A and B matrices. Let's take a closer look at this process.

LoRA fine-tunes large pretrained models by keeping the original weights W_0 frozen and learning only a low-rank approximation of the weight updates. Instead of directly training the full update matrix ΔW , LoRA factorizes it into two much smaller matrices A and B ($\Delta W=BA$). During inference the model output becomes $h=W_0x+Bx$. This approach drastically reduces the number of trainable parameters



Advantages

Parameter efficiency: Only a few million parameters are trained instead of billions.

Storage efficiency: You can store separate LoRA adapters for different tasks and swap them into the same base model.

Faster training: Lower memory footprint and less compute.

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	RI/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Limitations of LoRA

Task-Specific Adapters: You need a separate adapter for each task/domain, which can accumulate if you have many tasks.

Limited Parameter Update Scope : Only low-rank adapters are trained; for tasks requiring deep changes in the model's representation, this may underperform full fine-tuning.

Rank Selection is Crucial: Choosing a rank r too low can hurt accuracy, too high undermines the efficiency gains.

Quantized Low-Rank Adaptation (QLoRA)

QLoRA builds on LoRA by combining low-rank adapters with model quantization.

It fine-tunes large pretrained models while keeping the original weights frozen, but instead of using the full-precision weights of the base model, QLoRA loads the model in a low-precision (quantized) format (e.g. 4-bit).

The adapters are still trained in normal precision, but the base model stays frozen and quantized, making fine-tuning even more memory- and compute-efficient than LoRA alone.

Quantization:

Quantization is a technique that is helpful in reducing the size of the model by converting high precision data to low precision. In simple terms it converts datatype of high bits to fewer bits. For example, converting FP32 to 8-bit Integers is a quantization technique.

Why QLoRA Exists

Even with LoRA, loading a huge model in full precision can exceed GPU memory limits.

QLoRA solves this by:

Quantizing the base model (4-bit or 8-bit weights) to drastically cut memory usage.

Keeping adapters in higher precision for stable training.

Allowing fine-tuning of models with billions of parameters on a single GPU without sacrificing much accuracy.

Training Process:

The pretrained model is loaded with quantized 4-bit weights.

Only the LoRA adapters are updated during training.

Libraries like BitsAndBytes (for quantization) and PEFT (for LoRA) are used together for implementation.

Example code involves setting up quantization configs, enabling gradient checkpointing, and applying LoRA adapters to target modules (e.g., query, key, value layers).

How QLoRA Works

The pretrained weight matrix W_0 is first quantized (4-bit).

The weight update ΔW is still factorized into two low-rank matrices A and B: $\Delta W = BA$.

Only A and B (the adapters) are trained in higher precision; the quantized w_0 stays frozen.

During inference:

$$h = Q(W_0)x + BAx$$

where $Q(W_0)$ is the quantized base weight.

This combines the memory savings of quantization with the parameter efficiency of LoRA.

Advantages

Massive memory savings: quantized base model can be up to 4× smaller.

Parameter efficiency: still only trains small low-rank adapters.

Run huge models on modest hardware: fine-tune 65B+ parameter models on a single high-end GPU.

Almost no accuracy loss compared to full-precision LoRA.

Limitations of QLoRA

Quantization Overhead: Quantizing the base model can cause small but non-zero accuracy drops especially for sensitive tasks.

Training Stability: Combining quantized weights with high-precision adapters can require careful tuning of learning rate and scaling to remain stable.

Still Needs High-Precision Optimizer States: Optimizer states for adapters are usually stored in full precision, which can still consume noticeable memory for very large models.

Hardware Support: Not all GPUs/TPUs efficiently support low-precision (4-bit) matrix multiplications at high speed.

Feature	LoRA	QLoRA
Parameter reduction	Low-rank approximation of ΔW	Quantization of LoRA adapter weights
Memory footprint	Reduced	Further reduced
Fine-tuning speed	Fast	Slightly slower than LoRA
Performance	Close to traditional fine-tuning	Similar to LoRA

Choosing between LoRA and QLoRA:

If memory footprint is the primary concern: QLoRA is the better choice due to its even greater memory efficiency.

If fine-tuning speed is crucial: LoRA may be preferable due to its slightly faster training times.

If both memory and speed are important: QLoRA offers a good balance between both.

References

[Fine-Tuning using LoRA and QLoRA - GeeksforGeeks](#)

[Low-Rank Adapter \(LoRA\) Explained | by Sheli Kohan | Medium](#)

[Fine Tuning LLM with QLoRA | Medium](#)

[Difference between QLoRA and LoRA for Fine-Tuning LLMs. | by Sujatha Mudadla | Medium](#)